

MAT 335E – TERM PROJECT

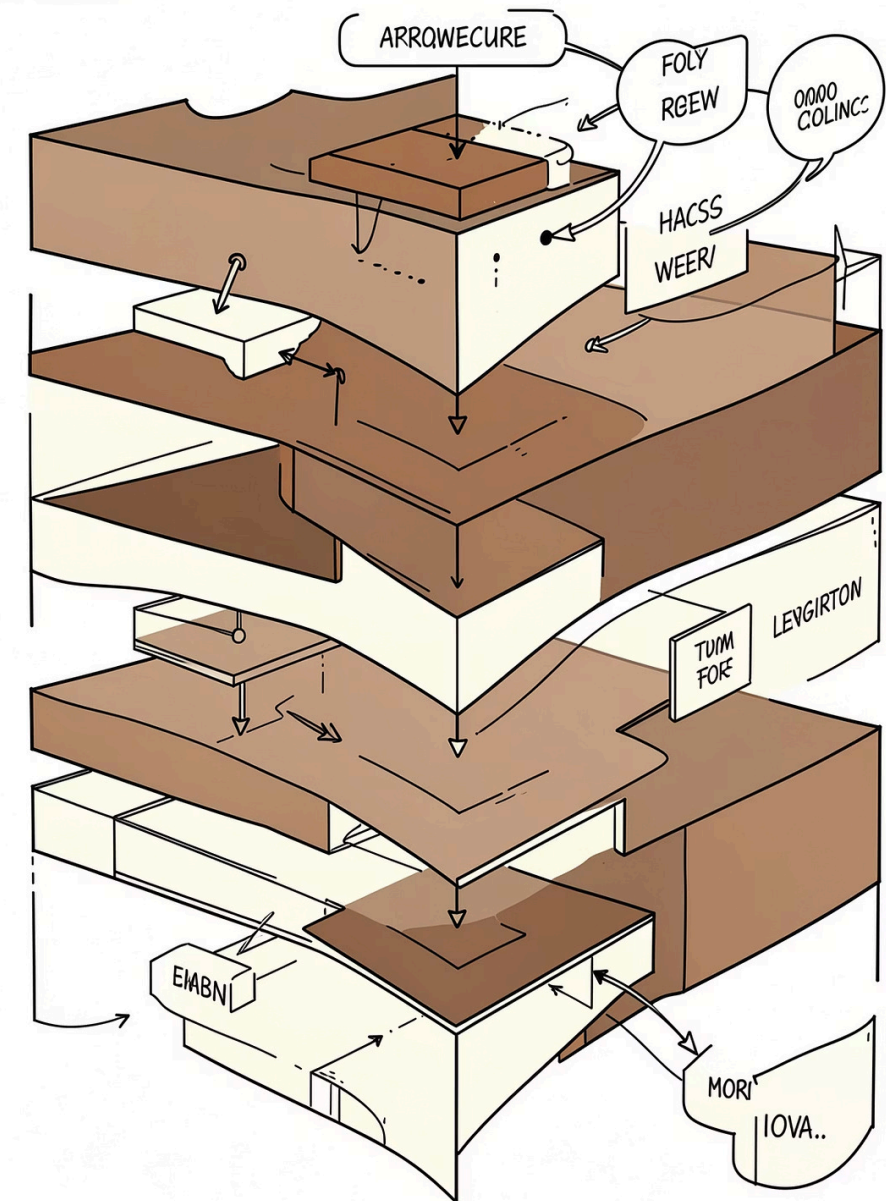
Fitness Tracker Application

A Native Android Workout Logging System

Developed by **Rahmet Sare Yılmaz** & **Kenan Güzel** for Istanbul Technical University, Department of Mathematical Engineering.



Made with GAMMA



Project Objectives & Architecture

Core Objectives

- Build a native Android fitness app in Java
- Apply OOP principles (Inheritance, Polymorphism)
- Manage athlete profiles & workouts via SQLite
- Provide a clean, validated user interface

Layered Architecture

- **Model Layer:** Plain Java classes (Athlete, Activity)
- **Data Layer:** DatabaseHelper managing SQLite CRUD
- **View Layer:** Android XML layouts for UI
- **Controller Layer:** Android Activities coordinating flow

Object-Oriented Domain Model

The core domain leverages inheritance and polymorphism to model different sports under a unified Activity structure.



Running Subclass

Inherits from Activity. Adds **distance** (km) and overrides `getSummary()` for running-specific metrics.



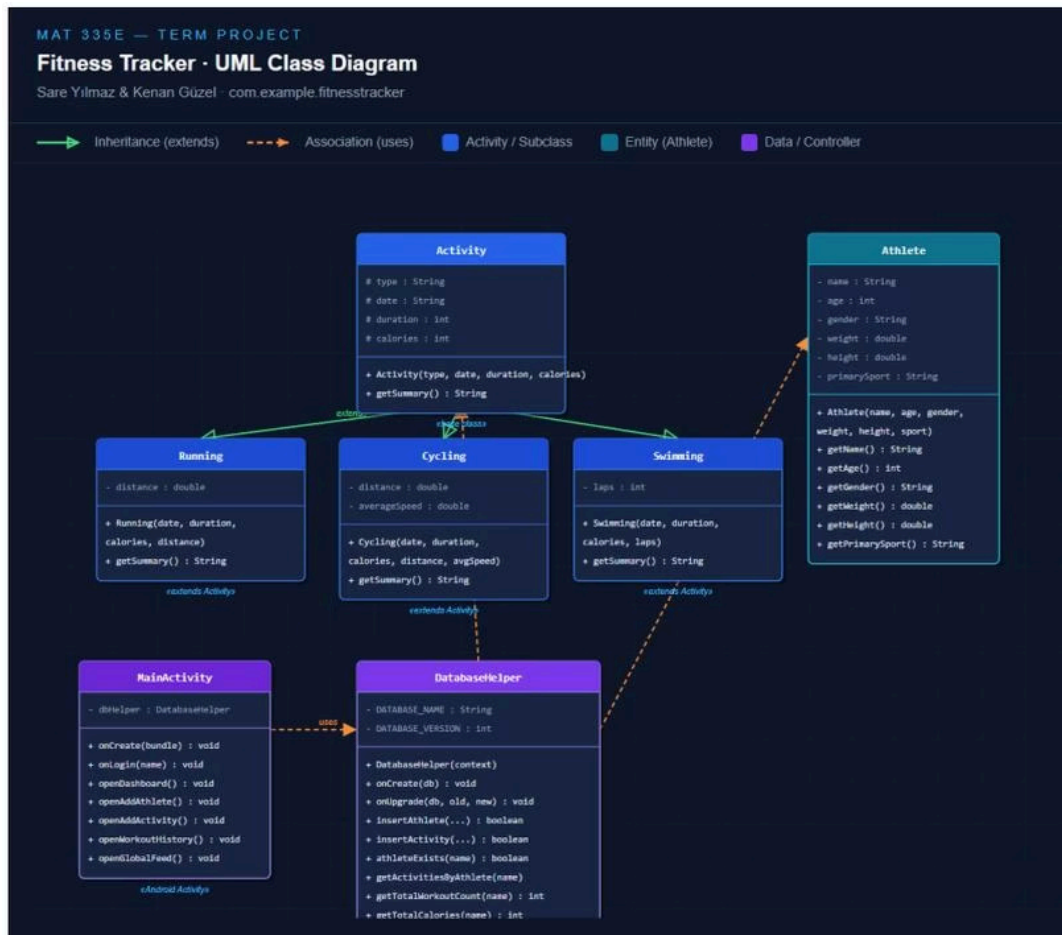
Cycling Subclass

Inherits from Activity. Adds **distance** and **averageSpeed** (km/h) to track cycling performance.



Swimming Subclass

Inherits from Activity. Adds **laps** count to track pool workouts accurately.



UML Class Diagram

The structural design of the application showing the inheritance hierarchy of activities and the controller relationships.

- **Inheritance:** Running, Cycling, and Swimming extend the base Activity class.
- **Association:** DatabaseHelper manages both Athlete and Activity records.
- **Controller:** MainActivity coordinates UI actions using DatabaseHelper.

OOP Principles

Encapsulation keeps fields private, while polymorphism enables dynamic summary generation via overridden methods.

Database Schema Design

The application utilizes a local SQLite database with two primary tables to persist athlete profiles and activity logs.

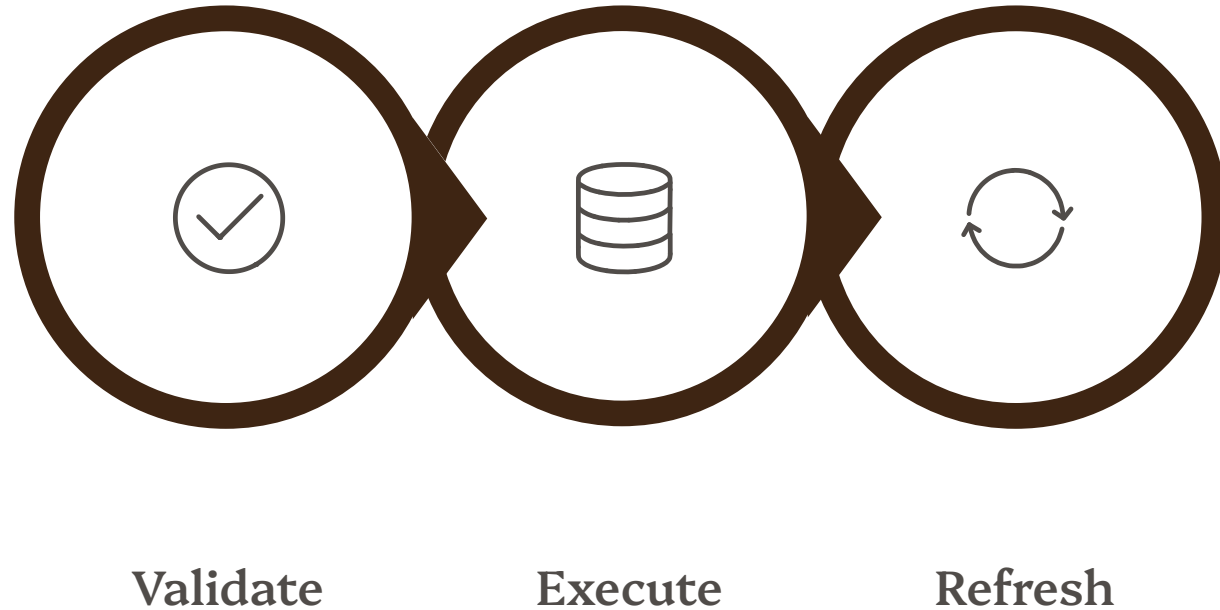
The 'athletes' Table

Column	Type	Description
id	INTEGER	Primary Key (Auto)
name	TEXT	Athlete's full name
primary_sport	TEXT	Preferred sport type

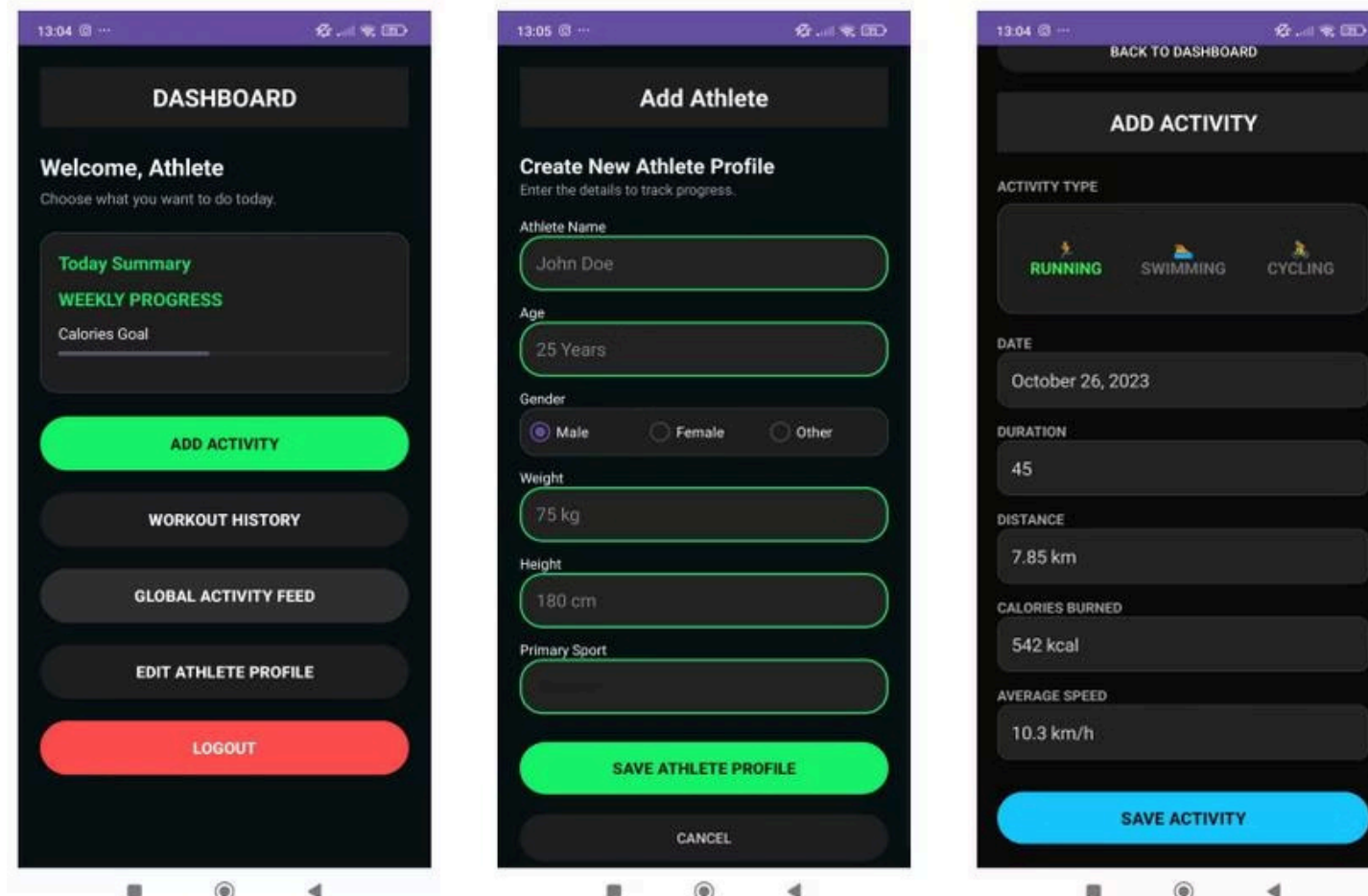
The 'activities' Table

Column	Type	Description
id	INTEGER	Primary Key (Auto)
athlete_name	TEXT	Foreign Key reference
type	TEXT	Running/Cycling/Swimming

Database Helper Operations



The DatabaseHelper class exposes robust public methods to handle all data operations safely across the application lifecycle.



User Interface: Athlete Screens

The application features seven distinct screens built with native Android XML layouts to guide the athlete's workflow.

Dashboard Screen

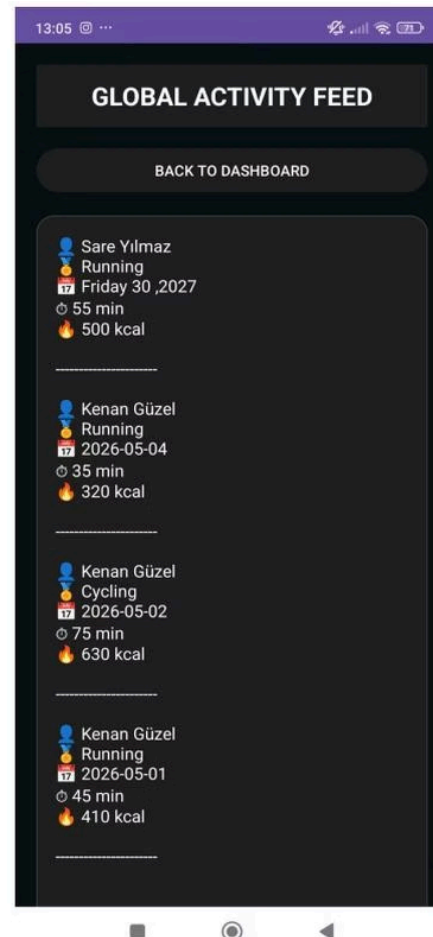
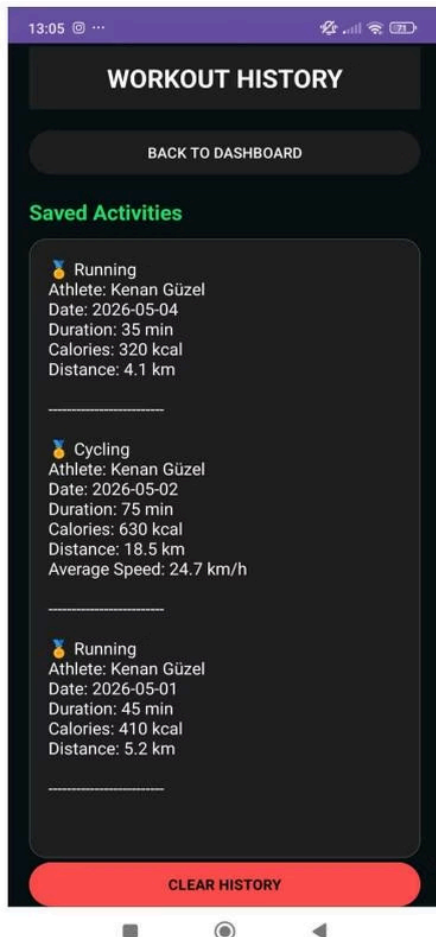
Displays weekly progress, total workouts, calories burned, and total active duration at a glance.

Add Athlete Profile

A registration form capturing name, age, gender, weight, height, and primary sport preferences.

Dynamic Add Activity

Form fields dynamically adapt (showing distance, laps, or speed) based on the selected sport type.



History & Global Activity Feed

Workout History

Lists all past activities for the logged-in athlete, sorted newest first. Includes an option to clear history and reset statistics.

Global Activity Feed

A community-focused feed displaying logged workouts from all registered athletes, encouraging social motivation and training transparency.

Robust Input Validation

To ensure data integrity and a smooth user experience, the application implements strict validation rules before writing to SQLite.



Duplicate Detection

Calls `athleteExists()` to prevent registering multiple profiles under the same name.



Required Field Checks

Validates all mandatory form fields, prompting users with Toast messages if data is missing.



Dynamic UI Adaptation

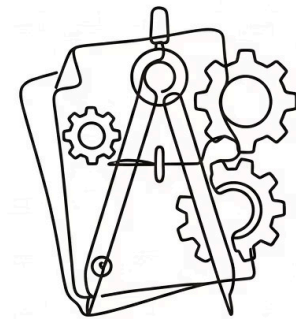
Hides or shows sport-specific fields on-the-fly to prevent invalid data entry for different activities.

Project Conclusion & Compliance

The Fitness Tracker successfully meets all academic requirements for MAT 335E, delivering a fully functional, object-oriented Android application.

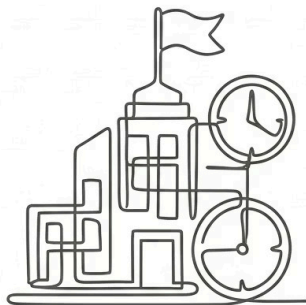
Requirement	Status
UML Class Diagram Consistency	100% Compliant
SQLite Database Integration	Fully Implemented
OOP Principles Applied	Verified

PROJECT INPUT REQUIREMENTS



Key deliverables, specifications, resources, and timelines are defined here.

FINAL IMPLEMENTATION STATUS



Complete structure, verified quality, and on-schedule delivery are documented.